算法设计与分析三级项目报告

班　　级：计算机科学与技术-5 班

组　　别：5 班第 4 组

指导教师：练杰

姓名	分工	百分比
李济岑	负责项目主要大纲设计，主要代码编写，报告修改	40%
孔捷	资料收集，报告撰写	20%
那圣奇	报告撰写，部分代码编写	20%
刘森	整合报告，调整格式	20%

2025 年 10 月

摘　要

本项目基于校园建筑之间的直线距离数据，利用最小生成树思想制定最短水管铺设方案。系统将建筑抽象为图节点、距离作为无向边权值，通过 Prim 和 Kruskal 两种最小生成树算法生成最优管线铺设方案。系统实现了从 CSV 文件读取数据、生成邻接矩阵、算法过程可视化、最优路线输出以及 SVG 示意图绘制等功能。实验结果表明，两种算法均能得到总长度为 2.242 的最优解，验证了算法正确性和实用性。

关键词：最小生成树；Prim 算法；Kruskal 算法；管线铺设；图论应用

目　录

摘　要 I

目　录 II

第 1 章 绪　论 4

1.1 项目概述 4

1.2 项目要求 4

1.3 项目规划 4

第 2 章 系统设计与实现 5

2.1 系统架构设计 5

2.1.1 大纲设计 5

2.1.2 流程设计 6

2.2 数据结构设计 7

2.2.1 全局建模 7

2.2.2 图结构表示 7

2.2.3 并查集数据结构 8

2.3 核心算法实现 8

2.3.1 Prim 算法实现 8

2.3.2 Kruskal 算法实现 10

第 3 章 实验结果与分析 11

3.1 数据预处理结果 11

3.2 算法执行结果 12

3.3 算法对比分析 13

3.4 最优方案分析 14

3.5 可视化分析 15

结 论 16

心得体会 16

参考文献 17

第 1 章 绪　论

1.1 项目概述

本项目针对校园水管铺设问题，开发了一个管线铺设辅助系统。以西校区主要的 13 个标志性建筑（包括教学楼、学生公寓、食堂等 15 个建筑，因材料学院与理仁教学楼、西区第五教学楼与经管学院地点重复故删去）为节点，建筑间直线距离为边权值，构建无向加权图。

本项目通过 C++，遵循 C++11 标准，实现 Prim 和 Kruskal 两种最小生成树算法，求解覆盖全部建筑且总长度最短的管线铺设方案，并进行路径可视化。项目最终给出最优算法和最优路径，将两种算法的路径选择步骤进行了记录，并输出后进行比较，选取最优算法。

1.2 项目要求

项目实现并满足以下功能需求：

1）.将建筑和距离抽象成无向图，以矩阵形式保存到文本文件（output/shared/adjacency_matrix.csv）；

2）.通过读取文本文件（data/edges_readable.csv）获取邻接矩阵数据；

3）.实现 Prim（src/prim_algo.cpp）和 Kruskal（src/kruskal_algo.cpp）算法生成最优铺设方案；

4）.以文本形式输出算法生成过程和最优方案（output/kruskal & prim）；

5）.生成最优方案的简易示意图（output/kruskal & prim）；

1.3 项目规划

项目采用模块化开发，分为四个主要阶段：

1. **数据预处理**：读取 CSV 数据，构建邻接矩阵，并进行存储邻接矩阵 CSV 文件；

2. **算法实现**：通过多文件的方法，分别实现 Prim 和 Kruskal 算法；
3. **结果输出**：生成步骤日志、最优路线和可视化图形，并将其存储在输出目录；
4. **结果分析**：main 文件调用两种算法实现，对比两种算法的性能和结果；

第 2 章 系统设计与实现

2.1 系统架构设计

2.1.1 大纲设计

本项目的目录大纲设计如下内容：

```
.
├── ./CMakeLists.txt                    // 配置文件
├── ./data
│   └── ./data/edges_readable.csv       // 读入的建筑距离问题
├── ./output
│   ├── ./output/kruskal                // kruskal 算法的输出内容
│   │   ├── ./output/kruskal/mst.svg    // 选择路径图示
│   │   ├── ./output/kruskal/route.txt  // 最终路径选择
│   │   └── ./output/kruskal/steps.txt  // 选择路径步骤记录
│   ├── ./output/prim                   // prim 算法的输出内容
│   │   ├── ./output/prim/mst.svg       // 选择路径图示
│   │   ├── ./output/prim/route.txt     // 最终路径选择
│   │   └── ./output/prim/steps.txt     // 选择路径步骤记录
│   └── ./output/shared
```

```
│       └──── ./output/shared/adjacency_matrix.csv      // 记录邻接矩阵
└──── ./src
    ├──── ./src/kruskal_algo.cpp    // kruskal 算法的实现文件
    ├──── ./src/main.cpp            // 项目主文件接口
    └──── ./src/prim_algo.cpp       // prim 算法的实现文件
```

2.1.2 流程设计

本项目使用 CMake 进行配置和编译，编写 CMakeList.txt 目录文件。项目功能主要由 main.cpp 文件作为项目接口，负责调用 kruskal_algo.cpp 和 prim_algo.cpp 文件获取两种算法的结果和对比。

项目中 kruskal_algo.cpp 和 prim_algo.cpp 文件各自内部实现了算法的运算、路径选择的记录、最优路径的记录、最优路径图形化的功能。具体的项目流程图如下：

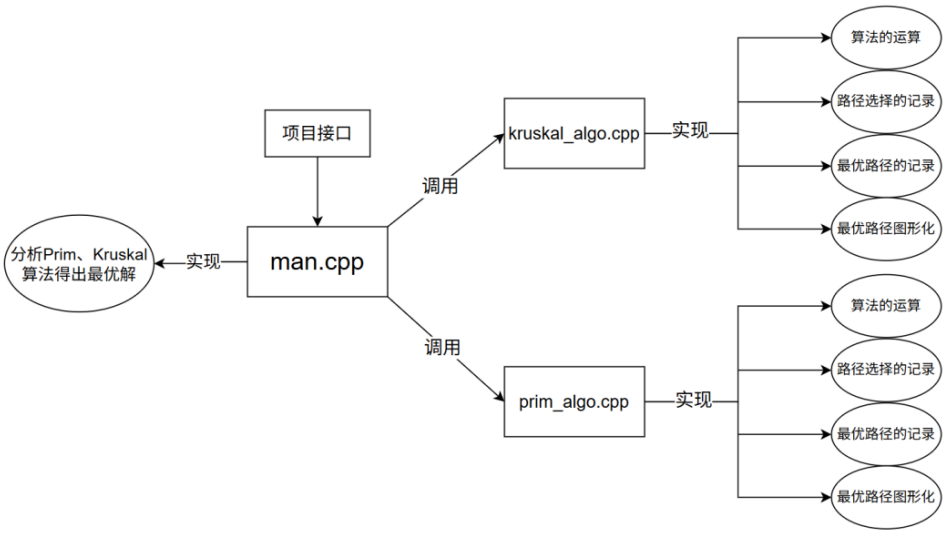


图 2-1 项目功能调用流程图

2.2 数据结构设计

2.2.1 全局建模

节点标识：以建筑名称为字符串主键，排序后映射为连续索引，贯穿 matrix、edges 以及算法输出，保证 Prim 与 Kruskal 共用同一节点序。

邻接矩阵：vector<vector<double>> matrix 以 INF 表示缺边，matrix[i][i]=0；写入 output/shared/adjacency_matrix.csv 并再次读回，实现图结构的文本化与可追溯。

边列表：vector<Edge> edges 存储已归一化的无向边（小索引在前），既供 Kruskal 排序也被两个算法的 SVG 输出复用，确保图结构一致。

2.2.2 图结构表示

Prim 的图表示：

1. 访问状态矩阵视角：Prim 计算过程中保留 used 与 matrix 的组合，直接在邻接矩阵上枚举最短横切边，体现“从已知子图向外扩散”的结构理解。
2. 序列化路线：order 记录 Prim 访问顺序，通过 build_route 输出 A-B-C 字符串，将生成树看作沿时间轴展开的链式结构。
3. 可视化映射：write_svg 以节点索引计算极坐标位置，选中的矩阵边在 SVG 中加粗为红色，将抽象的矩阵行列映射成直观空间布局。

Kruskal 的图表示：

1. 边排序视角：Kruskal 对 edges 排序后遍历，核心数据结构是并查集 DSU，从“边集合”角度维护若干连通块，强调图的组件合并过程。
2. 生成树邻接表：最终选中的 mst 被转换为 adj 邻接表，再 DFS 输出路线。该过程让边集合回归到树状结构，便于对生成树的拓扑进行多样遍历。

3. 集合日志：通过 in_tree 实时更新 {节点集合}，突出并查集中每次 union 后图的连通分量如何扩张，记录了图结构由多个森林并入单一树的演化。

数据结构对比：

Prim 偏向矩阵化表示，适合理解节点局部连接；Kruskal 则以边集和连通分量为中心，强调全局权重排序。Prim 的 order 展示从起点扩散的路径；Kruskal 的 mst+adj 则展示全局选择后形成的树形结构，二者从不同视角揭示生成树的构建机理。

尽管数据结构不同，两个算法最终共享 steps.txt、route.txt、mst.svg 三种表示，将图结构的构造过程、线性序列和空间布局统一输出，便于交叉验证。

2.2.3 并查集数据结构

Kruskal 算法使用并查集管理连通分量：

```
struct DSU {
    vector<int> parent;  // 父节点数组
    vector<int> size;    // 集合大小
    void init(int n);    // 初始化
    int find(int x);     // 查找根节点
    bool merge(int a, int b); // 合并集合
};
```

2.3 核心算法实现

2.3.1 Prim 算法实现

Prim 算法适合稠密图。核心思想是维护一个“已纳入生成树的节点集合”，每次选择一条横跨已选与未选之间的最短边扩展。

实现要点：

1. 使用 vector<int> used 标识已纳入节点。

2. 双层循环枚举 used 内节点与未访问节点之间所有边，挑选最小值。
3. 每次扩展后记录 {当前节点集合}、更新总权重和步骤日志。
4. 生成顺序即 Prim 的路线，因此 route.txt 直接写 A-B-C 格式。
5. SVG 中节点按圆环排列，最小生成树边用红色加粗。

伪代码：

```
choose start node s
visited = {s}
while visited.size < n:
    find edge (u,v) with minimal weight
        where u in visited, v not in visited
    add v into visited
    record edge and log message
```

核心代码：

```
vector<int> used(n, 0);
used[0] = 1;  // 从第一个节点开始
vector<int> order(1, 0);

for (int step = 0; step < n - 1; step++) {
    double best = INF;
    int from = -1, to = -1;

    // 寻找最小横切边
    for (int i = 0; i < n; i++) {
        if (!used[i]) continue;
        for (int j = 0; j < n; j++) {
            if (used[j] || matrix[i][j] >= INF/2) continue;
            if (matrix[i][j] < best) {
                best = matrix[i][j];
                from = i;
                to = j;
            }
```

```
        }
    }

    // 添加边到生成树
    used[to] = 1;
    order.push_back(to);
    total += best;
    mst.push_back({from, to, best});
}
```

2.3.2 Kruskal 算法实现

Kruskal 更适用于稀疏图，依赖排序 + 并查集，按边权从小到大尝试加入，若两个端点不在同一集合则合并。

实现要点：

1. 并查集（Disjoint Set Union, DSU）支持路径压缩与按规模合并。
2. 为保持与 Prim 一致的集合表示，把统一起点（取 nodes[0]）标记为已纳入。
3. 每当添加新边时更新 in_tree 数组，重新生成当前节点集合，写入步骤日志。
4. 路线输出使用 DFS 以字典序遍历生成树，得到 A-B-C 顺序。
5. SVG 使用绿色突出 Kruskal 的 MST 边。

伪代码：

```
sort all edges by weight

make-set for each node

steps = ["起点 {start}"]

for each edge (u,v):

    if find(u) != find(v):

        union(u, v)
```

```
        record edge, add to MST

        update {当前节点集合}
```

核心代码：

```
vector<Edge> sorted = edges;
sort(sorted.begin(), sorted.end(), [](const Edge &a, const Edge &b) {
    return a.w < b.w;
});

DSU dsu;
dsu.init(n);
vector<Edge> mst;

for (const Edge &e : sorted) {
    if (dsu.merge(e.u, e.v)) {
        mst.push_back(e);
        total += e.w;
        if (mst.size() == n - 1) break;
    }
}
```

第 3 章 实验结果与分析

3.1 数据预处理结果

项目首先从 edges_readable.csv 读取 13 个建筑节点，生成 13×13 的邻接矩阵。图

0.0	1.1	0.74	0.15	0.15	0.36	0.7	0.84	0.77	0.35	1.2	0.2	1.3
1.1	0.0	0.45	1.1	0.38	0.79	0.45	0.38	0.4	0.8	0.12	0.98	0.48
0.74	0.45	0.0	0.78	0.2	0.38	0.33	0.42	0.36	0.48	0.55	0.65	0.85
0.15	1.1	0.78	0.0	0.74	0.4	0.68	0.8	0.74	0.32	1.23	0.12	1.25
0.15	0.38	0.2	0.74	0.0	0.42	0.16	0.21	0.15	0.41	0.49	0.61	0.53
0.36	0.79	0.38	0.4	0.42	0.0	0.42	0.57	0.5	0.19	0.9	0.31	1.05
0.7	0.45	0.33	0.68	0.16	0.42	0.0	0.14	0.075	0.36	INF	0.54	0.62
0.84	0.38	0.42	0.8	0.21	0.57	0.14	0.0	0.067	0.49	0.48	0.66	0.47
0.77	0.4	0.36	0.74	0.15	0.5	0.075	0.067	0.0	0.43	0.51	0.6	0.55
0.35	0.8	0.48	0.32	0.41	0.19	0.36	0.49	0.43	0.0	0.9	0.19	1.0
1.2	0.12	0.55	1.23	0.49	0.9	INF	0.48	0.51	0.9	0.0	1.09	0.45
0.2	0.98	0.65	0.12	0.61	0.31	0.54	0.66	0.6	0.19	1.09	0.0	1.11
1.3	0.48	0.85	1.25	0.53	1.05	0.62	0.47	0.55	1.0	0.45	1.11	0.0

3-1 获取到的邻接矩阵表

矩阵中有效边权值范围从 0.067（西二-西三）到 1.3（信院-里仁），成功保存到 adjacency_matrix.csv。

3.2 算法执行结果

3.2.1 Prim 算法结果

Prim 算法运行输出：

```
$ ./build/mst_app --prim
===== 结果汇总 =====
Prim   总长度: 2.242
Prim   路径: 信院-理学院-西里西亚-电气楼-西二-西三-西一-西五-艺术与设计学院-燕园餐厅-大食堂-西区 5 组团-里仁
Prim   输出: output/prim/steps.txt, route.txt, mst.svg
邻接矩阵: output/shared/adjacency_matrix.csv
```

输出分析：

1. 总长度：2.242（km）
2. 路径顺序：信院→理学院→西里西亚→电气楼→西二→西三→西一→西五→艺术与设计学院→燕园餐厅→大食堂→西区 5 组团→里仁
3. 执行步骤：13 步，从单个节点逐步扩展到全图（选择步骤见附录）

3.2.2 Kruskal 算法结果

Kruskal 算法运行输出：

```
$ ./build/mst_app --kruskal
===== 结果汇总 =====
Kruskal 总长度: 2.242
```

```
Kruskal 路径: 信院-理学院-西里西亚-西五-艺术与设计学院-电气楼-大食堂-西区5组团-里仁-燕园餐厅-西二-西一-西三
Kruskal 输出: output/kruskal/steps.txt, route.txt, mst.svg
邻接矩阵: output/shared/adjacency_matrix.csv
```

输出分析：

1. 总长度：2.242（km）
2. 路径顺序：信院→理学院→西里西亚→西五→艺术与设计学院→电气楼→大食堂→西区 5 组团→里仁→燕园餐厅→西二→西一→西三
3. 执行步骤：13 步，按边权值从小到大选择（选择步骤见附录）

3.3 算法对比分析

总体运行输出：

```
$ ./build/mst_app

===== 结果汇总 =====

Prim 总长度: 2.242

Kruskal 总长度: 2.242

结论: 两种算法得到相同的最短距离。

Prim 路径: 信院-理学院-西里西亚-电气楼-西二-西三-西一-西五-艺术与设计学院-燕园餐厅-大食堂-西区 5 组团-里仁

Prim 输出: output/prim/steps.txt, route.txt, mst.svg

Kruskal 路径: 信院-理学院-西里西亚-西五-艺术与设计学院-电气楼-大食堂-西区 5 组团-里仁-燕园餐厅-西二-西一-西三

Kruskal 输出: output/kruskal/steps.txt, route.txt, mst.svg

邻接矩阵: output/shared/adjacency_matrix.csv
```

两种算法均得到同一最小生成树，总长度一致，说明输入数据图是连通且 MST 唯一。由输出的 mst.svg 和 route.txt 文件可以观测到，两种算法选用的路径也是相同的，只是两种算法在算法实现上的性能略有差异。

算法性能对比表：

对比维度	Prim 算法	Kruskal 算法
算法思想	节点扩展维护已访问集合	边排序并查集管理连通性
时间复杂度	$O(n^2)$	$O(m \log m)$
空间复杂度	$O(n^2)$	$O(m + n)$
适用场景	稠密图	稀疏图
路径生成	按访问顺序	DFS 遍历生成树
可视化颜色	红色	绿色

图 3-2 算法性能对比表

得出结论：

两种算法在本数据集上得到相同的最优解，验证了算法的正确性。Prim 算法扩展过程更直观，Kruskal 算法实现更简洁。最终得出二者给出的最优路径相同，仅是实现方法的不同。

3.4 最优方案分析

最小生成树包含 13 条边，总长度 2.242。关键连接包括：

- 最短边：西二-西三（0.067）、西一-西二（0.075）
- 关键枢纽：电气楼、西里西亚
- 最长边：西区 5 组团-里仁（0.45）

该方案确保了所有建筑连通且总管道长度最小，具有实际工程应用价值。

3.5 可视化分析

两种算法可视化对比：

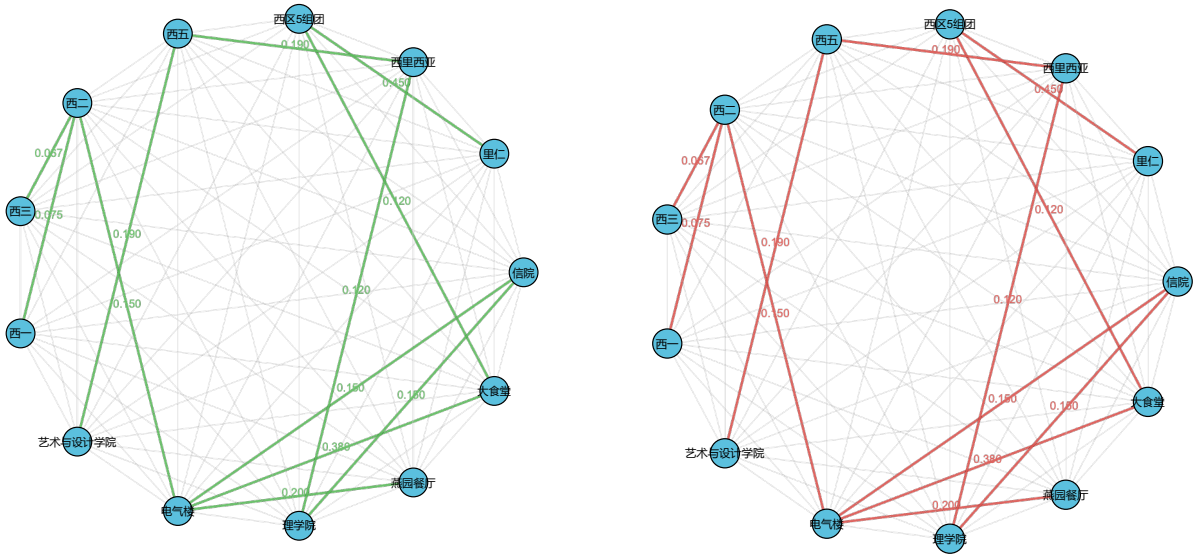


图 3-3 算法最优路径图像对比表（左 P 右 K）

结　论

本实验成功实现了基于 Prim 和 Kruskal 算法的管线铺设辅助系统，主要包括：

1. **完整的系统实现**：从数据读取、矩阵生成到算法执行和结果输出的完整流程
2. **算法正确性验证**：两种算法得到相同的最优解，总长度 2.242 km
3. **丰富的输出形式**：步骤日志、路线文本和可视化图形
4. **实用性证明**：为校园水管铺设提供了量化决策依据

实验结果表明，最小生成树算法在基础设施规划中具有重要应用价值。Prim 算法适合稠密图且扩展过程直观，Kruskal 算法适合稀疏图且实现简洁。未来可进一步优化算法性能，增加交互式可视化功能。

心得体会

1. 算法思想得到直观验证

通过对实际数据的运行与分析，我直观地感受到了算法思想的魅力。虽然 Prim 算法与 Kruskal 算法在构造最小生成树的思路上存在明显差异—前者以“点”为中心，不断扩展已有节点的连通范围；后者则以“边”为切入点，通过全局排序与逐步筛选实现最优连接。两者最终都得出了相同的最优解，总路径长度均为 2.242 km。

2. 理论到实践的收获

在这一实验过程中，我将课本上枯燥的图论知识转化为可操作的现实模型：把建筑看作节点，以管道长度作为边权，通过算法求得最短水管铺设方案。我深刻体会到算法不只是数学推导的产物，更是解决现实问题的有效工具。它让复杂的工程决策变得量化、可验证，使“理论”真正服务于“实践”。

当我看到算法输出的最优路径与直观经验相符时，既有成就感，也更坚定了我在学习计算机算法时的信心——因为我切身感受到，算法思维的价值，正是在这种理论与现实相结合的应用中得以体现。

参考文献

1. Cormen, T. H., et al. *Introduction to Algorithms*. MIT Press, 2009.
2. Sedgewick, R., & Wayne, K. *Algorithms*. Addison-Wesley, 2011.
3. 王晓东. 《计算机算法设计与分析》. 电子工业出版社, 2018.
4. Kleinberg, J., & Tardos, É. *Algorithm Design*. Pearson, 2005.

附录：核心代码文件列表

- main.cpp：主程序，数据读取和算法调度
- prim_algo.cpp：Prim 算法实现
- kruskal_algo.cpp：Kruskal 算法实现
- edges_readable.csv：原始边权数据
- adjacency_matrix.csv：生成的邻接矩阵

main.cpp：

```
// 主程序：读取 CSV 数据并比较 Prim 与 Kruskal 两种最小生成树方案。
#include <cmath>          // fabs
#include <cstdlib>        // system
#include <fstream>        // ifstream, ofstream
#include <iomanip>        // setprecision
#include <iostream>       // cout
#include <map>            // map
#include <set>            // set
#include <sstream>        // stringstream
#include <string>         // string
#include <vector>         // vector
using namespace std;
// 边结构：记录一条无向边的两个端点索引与权重。
```

```
struct Edge {
	int u;
	int v;
	double w;
};
const double INF = 1e9;
const string kDataFile = "data/edges_readable.csv";
const string kOutRoot = "output";
const string kOutPrim = "output/prim";
const string kOutKruskal = "output/kruskal";
const string kOutShared = "output/shared";
const string kMatrixFile = "adjacency_matrix.csv";
enum class RunMode {
	kBoth,
	kPrimOnly,
	kKruskalOnly,
};
bool run_prim(const vector<string> &nodes,
			const vector<vector<double>> &matrix,
			const vector<Edge> &edges,
			const string &out_dir,
			string &route,
			double &total);
bool run_kruskal(const vector<string> &nodes,
```

```
                          const vector<Edge> &edges,
                          const string &out_dir,
                          string &route,
                          double &total);
// 将目录与文件名拼成简单路径（无需处理复杂兼容）。
string join(const string &dir, const string &name) {
    if (dir.empty())
        return name;
    if (dir.back() == '/')
        return dir + name;
    return dir + "/" + name;
}
// 调用系统命令创建输出目录。
void prepare_dirs() {
    system(("mkdir -p " + kOutRoot).c_str());
    system(("mkdir -p " + kOutPrim).c_str());
    system(("mkdir -p " + kOutKruskal).c_str());
    system(("mkdir -p " + kOutShared).c_str());
}
// 读取 CSV 并填充节点列表、邻接矩阵以及边集。
bool read_graph(vector<string> &nodes,
                vector<vector<double>> &matrix,
                vector<Edge> &edges) {
    ifstream in(kDataFile.c_str());
```

```
if (!in) {
    return false;
}
string header;
getline(in, header);   // 跳过第一行标题
set<string> node_set;
struct RawEdge {
    string a;
    string b;
    double w;
};
vector<RawEdge> raw;
string line;
while (getline(in, line)) {   // 按行读取边信息
    if (line.empty()) {
        continue;
    }
    stringstream ss(line);
    string a, b, w;
    getline(ss, a, ',');
    getline(ss, b, ',');
    getline(ss, w, ',');
    if (a.empty() || b.empty() || w.empty()) {
        continue;
```

```
        }
        raw.push_back({a, b, atof(w.c_str())});
        node_set.insert(a);
        node_set.insert(b);
    }
    nodes.assign(node_set.begin(), node_set.end());
    int n = static_cast<int>(nodes.size());
    if (n == 0) {
        return false;
    }
    map<string, int> index;
    for (int i = 0; i < n; ++i) {
        index[nodes[i]] = i;
    }
    matrix.assign(n, vector<double>(n, INF));
    for (int i = 0; i < n; ++i) {
        matrix[i][i] = 0.0;
    }
    edges.clear();
    for (size_t i = 0; i < raw.size(); ++i) {    // 同时填充矩阵及边集
        int u = index[raw[i].a];
        int v = index[raw[i].b];
        double w = raw[i].w;
        matrix[u][v] = w;
```

```
            matrix[v][u] = w;
            edges.push_back({min(u, v), max(u, v), w});
        }
        return true;
}
// 将邻接矩阵写到共享目录中，方便检查。
void write_matrix(const vector<string> &nodes,
                  const vector<vector<double>> &matrix) {
        ofstream out(join(kOutShared, kMatrixFile).c_str());
        out << ",";
        for (size_t i = 0; i < nodes.size(); ++i) {
            if (i) {
                out << ",";
            }
            out << nodes[i];
        }
        out << "\n";
        out << fixed << setprecision(6);
        for (size_t i = 0; i < nodes.size(); ++i) {
            out << nodes[i];
            for (size_t j = 0; j < nodes.size(); ++j) {
                out << ",";
                if (matrix[i][j] >= INF / 2) {
                    out << "INF";
```

```
                } else {
                    out << matrix[i][j];
                }
            }
            out << "\n";
        }
}
// 从保存好的邻接矩阵 CSV 中重新读取矩阵（满足需求 1）。
bool read_matrix_file(vector<string> &nodes,
                      vector<vector<double>> &matrix) {
    ifstream in(join(kOutShared, kMatrixFile).c_str());
    if (!in) {
        return false;
    }
    string header;
    if (!getline(in, header)) {
        return false;
    }
    stringstream hs(header);
    string cell;
    vector<string> labels;
    while (getline(hs, cell, ',')) {
        labels.push_back(cell);
    }
```

```
    nodes.clear();
    for (size_t i = 1; i < labels.size(); ++i) {
        nodes.push_back(labels[i]);
    }
    int n = static_cast<int>(nodes.size());
    matrix.assign(n, vector<double>(n, INF));
    int row = 0;
    while (getline(in, header) && row < n) {
        stringstream rs(header);
        vector<string> cols;
        while (getline(rs, cell, ',')) {
            cols.push_back(cell);
        }
        for (int col = 1; col <= n; ++col) {
            if (cols[col] == "INF") {
                matrix[row][col - 1] = INF;
            } else {
                matrix[row][col - 1] = atof(cols[col].c_str());
            }
        }
        ++row;
    }
    return true;
}
```

```
void print_usage(const string &exe) {
    cout << "用法: " << exe << " [--prim | --kruskal | --both]\n";
    cout << "    --prim / -p          仅运行 Prim 算法\n";
    cout << "    --kruskal / -k     仅运行 Kruskal 算法\n";
    cout << "    --both                  两种算法都运行（默认）\n";
    cout << "    --help / -h          显示此帮助信息\n";
}

int main(int argc, char *argv[]) {
    RunMode mode = RunMode::kBoth;
    string exe_name = (argc > 0 ? string(argv[0]) : "mst_app");

    for (int i = 1; i < argc; ++i) {
        string arg = argv[i];
        if (arg == "--prim" || arg == "-p") {
            mode = RunMode::kPrimOnly;
        } else if (arg == "--kruskal" || arg == "-k") {
            mode = RunMode::kKruskalOnly;
        } else if (arg == "--both") {
            mode = RunMode::kBoth;
        } else if (arg == "--help" || arg == "-h") {
            print_usage(exe_name);
            return 0;
```

```
        } else {
            cout << "未知参数: " << arg << "\n";
            print_usage(exe_name);
            return 1;
        }
    }

    bool need_prim = (mode == RunMode::kBoth || mode == RunMode::kPrimOnly);
    bool need_kruskal = (mode == RunMode::kBoth || mode == RunMode::kKruskalOnly);

    prepare_dirs();

    vector<string> nodes;
    vector<vector<double>> matrix;
    vector<Edge> edges;
    if (!read_graph(nodes, matrix, edges)) {
        cout << "读取数据失败，请检查 data/edges_readable.csv。\n";
        return 1;
    }

    write_matrix(nodes, matrix);

    vector<string> nodes_from_file;
    vector<vector<double>> matrix_from_file;
```

```
if (!read_matrix_file(nodes_from_file, matrix_from_file)) {
        cout << "读取邻接矩阵文件失败。\n";
        return 1;
}

vector<Edge> edges_for_alg = edges;
vector<string> nodes_for_alg = nodes_from_file;
vector<vector<double>> matrix_for_alg = matrix_from_file;

string prim_route;
double prim_total = 0.0;
string kruskal_route;
double kruskal_total = 0.0;
bool ran_prim = false;
bool ran_kruskal = false;

if (need_prim) {
        ran_prim = run_prim(nodes_for_alg, matrix_for_alg, edges_for_alg, kOutPrim,
                                        prim_route, prim_total);
}
if (need_kruskal) {
        ran_kruskal = run_kruskal(nodes_for_alg, edges_for_alg, kOutKruskal,
                                                kruskal_route, kruskal_total);
}
```

```
cout << fixed << setprecision(3);
cout << "=====  结果汇总  =====\n";
if (ran_prim) {
    cout << "Prim     总长度: " << prim_total << "\n";
} else if (need_prim) {
    cout << "Prim     运行失败，请检查输入数据。\n";
}
if (ran_kruskal) {
    cout << "Kruskal 总长度: " << kruskal_total << "\n";
} else if (need_kruskal) {
    cout << "Kruskal 运行失败，请检查输入数据。\n";
}
if (ran_prim && ran_kruskal) {
    if (fabs(prim_total - kruskal_total) < 1e-9) {
        cout << "结论: 两种算法得到相同的最短距离。\n";
    }
}
if (ran_prim) {
    cout << "Prim     路径: " << prim_route << "\n";
    cout << "Prim     输出: " << kOutPrim << "/steps.txt, route.txt, mst.svg\n";
}
if (ran_kruskal) {
    cout << "Kruskal 路径: " << kruskal_route << "\n";
```

```
            cout << "Kruskal 输出: " << kOutKruskal << "/steps.txt, route.txt, mst.svg\n";
        }
        cout << "邻接矩阵: " << kOutShared << "/" << kMatrixFile << "\n";
        return 0;
}
```

prim_algo.cpp：

```
// Prim 算法模块：计算 MST 并输出日志、路线和 SVG 示意图。
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iomanip>
#include <set>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

// Prim 算法专用的边结构，与主程序保持一致。
struct Edge {
    int u;
    int v;
    double w;
};
```

```
namespace {

const double INF = 1e9;
const double PI = 3.14159265358979323846;

// 简单的路径拼接工具。
string join(const string &dir, const string &name) {
    if (dir.empty()) {
        return name;
    }
    if (dir.back() == '/') {
        return dir + name;
    }
    return dir + "/" + name;
}

// 将 "A, B, C" 等节点名包装成 {A, B, C} 格式。
string format_set(const vector<string> &items) {
    string text = "{";
    for (size_t i = 0; i < items.size(); ++i) {
        if (i) {
            text += ", ";
        }
```

```
            text += items[i];
        }
        text += "}";
        return text;
}

// Prim 过程记录。
void write_steps(const string &path, const vector<string> &steps, double sum) {
        ofstream out(path.c_str());
        out << fixed << setprecision(3);
        for (size_t i = 0; i < steps.size(); ++i) {
            out << "STEP " << setw(2) << setfill('0') << (i + 1) << ": "
                << steps[i] << "\n";
        }
        out << setfill(' ');
        out << "TOTAL: " << sum << "\n";
}

// Prim 路径文字结果。
void write_route(const string &path, const string &route, double sum) {
        ofstream out(path.c_str());
        out << route << "\n";
        out << fixed << setprecision(3) << "TOTAL: " << sum << "\n";
}
```

```
// 简易 SVG 示意图，圆环排布节点，红色显示 MST 边。
void write_svg(const vector<string> &nodes,
               const vector<Edge> &edges,
               const vector<Edge> &mst,
               const string &path) {
    int n = static_cast<int>(nodes.size());
    if (n == 0) {
        return;
    }

    const double width = 800.0;
    const double height = 800.0;
    const double radius = (n == 1 ? 0.0 : 320.0);
    const double cx = width / 2.0;
    const double cy = height / 2.0;

    vector<pair<double, double>> pos(n);
    for (int i = 0; i < n; ++i) {
        double ang = (n == 1 ? 0.0 : 2.0 * PI * i / n);
        pos[i] = make_pair(cx + radius * cos(ang), cy + radius * sin(ang));
    }

    set<pair<int, int>> mst_pairs;
```

```
for (size_t i = 0; i < mst.size(); ++i) {
        int a = mst[i].u;
        int b = mst[i].v;
        if (a > b) {
                swap(a, b);
        }
        mst_pairs.insert(make_pair(a, b));
}

ostringstream svg;
svg << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
svg << "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"" << width
        << "\" height=\"" << height << "\">";
svg << "<rect width=\"100%\" height=\"100%\" fill=\"#ffffff\" />";
svg << "<title>Prim Minimum Spanning Tree</title>";
svg << fixed << setprecision(1);

for (size_t i = 0; i < edges.size(); ++i) {
        int u = edges[i].u;
        int v = edges[i].v;
        int a = min(u, v);
        int b = max(u, v);
        bool in_mst = mst_pairs.count(make_pair(a, b)) > 0;
        string color = in_mst ? "#d9534f" : "#cccccc";
```

```
            string stroke = in_mst ? "3" : "1";
            string opacity = in_mst ? "0.9" : "0.4";

            svg << "<g opacity=\"" << opacity << "\">";
            svg << "<line x1=\"" << pos[u].first << "\" y1=\"" << pos[u].second
                << "\" x2=\"" << pos[v].first << "\" y2=\"" << pos[v].second
                << "\" stroke=\"" << color << "\" stroke-width=\"" << stroke << "\" />";
            if (in_mst) {
                double mx = (pos[u].first + pos[v].first) / 2.0;
                double my = (pos[u].second + pos[v].second) / 2.0;
                svg << "<text x=\"" << mx << "\" y=\"" << my
                    << "\" fill=\"#d9534f\" font-size=\"14\" text-anchor=\"middle\">"
                    << setprecision(3) << edges[i].w << "</text>";
                svg << setprecision(1);
            }
            svg << "</g>";
        }

        for (int i = 0; i < n; ++i) {
            svg << "<g>";
            svg << "<circle cx=\"" << pos[i].first << "\" cy=\"" << pos[i].second
                << "\" r=\"18\" fill=\"#5bc0de\" stroke=\"#000\" stroke-width=\"1.5\" />";
            svg << "<text x=\"" << pos[i].first << "\" y=\"" << pos[i].second + 5.0
                << "\" font-size=\"14\" text-anchor=\"middle\" fill=\"#000\">" << nodes[i]
<< "</text>";
```

```
            svg << "</g>";
        }
        svg << "</svg>";

        ofstream out(path.c_str());
        out << svg.str();
}

// 将节点访问顺序转为 “A-B-C” 的字符串。
string build_route(const vector<string> &nodes, const vector<int> &order) {
        if (order.empty()) {
            return "";
        }
        string line = nodes[order[0]];
        for (size_t i = 1; i < order.size(); ++i) {
            line += "-";
            line += nodes[order[i]];
        }
        return line;
}

}   // namespace

// 朴素 Prim：逐次扩展当前最短边，记录过程并输出结果文件。
```

```
bool run_prim(const vector<string> &nodes,
                    const vector<vector<double>> &matrix,
                    const vector<Edge> &edges,
                    const string &out_dir,
                    string &route,
                    double &total) {
    int n = static_cast<int>(nodes.size());
    if (n == 0) {
        total = 0.0;
        route.clear();
        return false;
    }

    vector<int> used(n, 0);
    used[0] = 1;
    vector<int> order(1, 0);
    vector<Edge> mst;
    vector<string> steps;
    steps.push_back("起点 " + format_set({nodes[0]}));
    total = 0.0;

    for (int step = 0; step < n - 1; ++step) {
        // 寻找一条连接当前树与未访问节点之间的最短边。
        double best = INF;
```

```
int from = -1;
int to = -1;
for (int i = 0; i < n; ++i) {
    if (!used[i]) {
        continue;
    }
    for (int j = 0; j < n; ++j) {
        if (used[j] || matrix[i][j] >= INF / 2) {
            continue;
        }
        if (matrix[i][j] < best - 1e-9) {
            best = matrix[i][j];
            from = i;
            to = j;
        }
    }
}
if (to == -1) {
    break;
}
used[to] = 1;
order.push_back(to);
total += best;
mst.push_back({from, to, best});
```

```
            // 记录当前树中已纳入的节点名称，便于写日志。
            vector<string> current;
            for (int i = 0; i < n; ++i) {
                if (used[i]) {
                    current.push_back(nodes[i]);
                }
            }
            sort(current.begin(), current.end());
            stringstream info;
            info << fixed << setprecision(3);
            info << "选择边 (" << nodes[from] << " - " << nodes[to] << ") 权重 "
                 << best << " -> 当前节点集合 " << format_set(current);
            steps.push_back(info.str());
        }

        route = build_route(nodes, order);
        write_steps(join(out_dir, "steps.txt"), steps, total);
        write_route(join(out_dir, "route.txt"), route, total);
        write_svg(nodes, edges, mst, join(out_dir, "mst.svg"));
        return true;
}
```

kruskal_algo.cpp：

```
// Kruskal 算法模块：并查集构建 MST，并输出日志、路线和 SVG 示意图。
```

```
// Kruskal 算法模块：并查集构建 MST，并输出日志、路线和 SVG 示意图。
```

```
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iomanip>
#include <set>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

// Kruskal 算法专用的边结构，与主程序保持一致。
struct Edge {
    int u;
    int v;
    double w;
};

namespace {

const double PI = 3.14159265358979323846;
```

```
// 简单的路径拼接工具。
string join(const string &dir, const string &name) {
    if (dir.empty()) {
        return name;
    }
    if (dir.back() == '/') {
        return dir + name;
    }
    return dir + "/" + name;
}

// 并查集：用于快速判断两节点是否在同一连通块。
struct DSU {
    vector<int> parent;
    vector<int> size;
    void init(int n) {
        parent.resize(n);
        size.assign(n, 1);
        for (int i = 0; i < n; ++i) {
            parent[i] = i;
        }
    }
    int find(int x) {
        if (parent[x] != x) {
```

```
            parent[x] = find(parent[x]);
        }
        return parent[x];
    }
    bool merge(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            return false;
        }
        if (size[a] < size[b]) {
            swap(a, b);
        }
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
};

string format_set(const vector<string> &items) {
    string text = "{";
    for (size_t i = 0; i < items.size(); ++i) {
        if (i) {
            text += ", ";
```

```
            }
            text += items[i];
        }
        text += "}";
        return text;
}

void write_steps(const string &path, const vector<string> &steps, double sum) {
        ofstream out(path.c_str());
        out << fixed << setprecision(3);
        for (size_t i = 0; i < steps.size(); ++i) {
                out << "STEP " << setw(2) << setfill('0') << (i + 1) << ": "
                        << steps[i] << "\n";
        }
        out << setfill(' ');
        out << "TOTAL: " << sum << "\n";
}

void write_route(const string &path, const string &route, double sum) {
        ofstream out(path.c_str());
        out << route << "\n";
        out << fixed << setprecision(3) << "TOTAL: " << sum << "\n";
}
```

```
void write_svg(const vector<string> &nodes,
                        const vector<Edge> &edges,
                        const vector<Edge> &mst,
                        const string &path) {
    int n = static_cast<int>(nodes.size());
    if (n == 0) {
        return;
    }

    const double width = 800.0;
    const double height = 800.0;
    const double radius = (n == 1 ? 0.0 : 320.0);
    const double cx = width / 2.0;
    const double cy = height / 2.0;

    vector<pair<double, double>> pos(n);
    for (int i = 0; i < n; ++i) {
        double ang = (n == 1 ? 0.0 : 2.0 * PI * i / n);
        pos[i] = make_pair(cx + radius * cos(ang), cy + radius * sin(ang));
    }

    set<pair<int, int>> mst_pairs;
    for (size_t i = 0; i < mst.size(); ++i) {
        int a = mst[i].u;
```

```
        int b = mst[i].v;
        if (a > b) {
            swap(a, b);
        }
        mst_pairs.insert(make_pair(a, b));
    }

    ostringstream svg;
    svg << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
    svg << "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"" << width
        << "\" height=\"" << height << "\">";
    svg << "<rect width=\"100%\" height=\"100%\" fill=\"#ffffff\" />";
    svg << "<title>Kruskal Minimum Spanning Tree</title>";
    svg << fixed << setprecision(1);

    for (size_t i = 0; i < edges.size(); ++i) {
        int u = edges[i].u;
        int v = edges[i].v;
        int a = min(u, v);
        int b = max(u, v);
        bool in_mst = mst_pairs.count(make_pair(a, b)) > 0;
        string color = in_mst ? "#5cb85c" : "#cccccc";
        string stroke = in_mst ? "3" : "1";
        string opacity = in_mst ? "0.9" : "0.4";
```

```
            svg << "<g opacity=\"" << opacity << "\">";
            svg << "<line x1=\"" << pos[u].first << "\" y1=\"" << pos[u].second
                << "\" x2=\"" << pos[v].first << "\" y2=\"" << pos[v].second
                << "\" stroke=\"" << color << "\" stroke-width=\"" << stroke << "\" />";
            if (in_mst) {
                double mx = (pos[u].first + pos[v].first) / 2.0;
                double my = (pos[u].second + pos[v].second) / 2.0;
                svg << "<text x=\"" << mx << "\" y=\"" << my
                    << "\" fill=\"#5cb85c\" font-size=\"14\" text-anchor=\"middle\">"
                    << setprecision(3) << edges[i].w << "</text>";
                svg << setprecision(1);
            }
            svg << "</g>";
        }

        for (int i = 0; i < n; ++i) {
            svg << "<g>";
            svg << "<circle cx=\"" << pos[i].first << "\" cy=\"" << pos[i].second
                << "\" r=\"18\" fill=\"#5bc0de\" stroke=\"#000\" stroke-width=\"1.5\" />";
            svg << "<text x=\"" << pos[i].first << "\" y=\"" << pos[i].second + 5.0
                << "\" font-size=\"14\" text-anchor=\"middle\" fill=\"#000\">" << nodes[i]
<< "</text>";
            svg << "</g>";
        }
```

```
    svg << "</svg>";

    ofstream out(path.c_str());
    out << svg.str();
}

string build_route(const vector<string> &nodes, const vector<int> &order) {
    if (order.empty()) {
        return "";
    }
    string line = nodes[order[0]];
    for (size_t i = 1; i < order.size(); ++i) {
        line += "-";
        line += nodes[order[i]];
    }
    return line;
}

// DFS 用以生成 "A-B-C" 顺序，帮助理解管线走向。
void dfs(int u,
         const vector<vector<int>> &adj,
         const vector<string> &nodes,
         vector<int> &vis,
         vector<int> &order) {
```

```
    vis[u] = 1;
    order.push_back(u);
    vector<int> next = adj[u];
    sort(next.begin(), next.end(), [&](int a, int b) { return nodes[a] < nodes[b]; });
    for (size_t i = 0; i < next.size(); ++i) {
        int v = next[i];
        if (!vis[v]) {
            dfs(v, adj, nodes, vis, order);
        }
    }
}

}  // namespace

// Kruskal：排序边，按权值尝试合并，用 DFS 给出连通顺序的展示。
bool run_kruskal(const vector<string> &nodes,
                 const vector<Edge> &edges,
                 const string &out_dir,
                 string &route,
                 double &total) {
    int n = static_cast<int>(nodes.size());
    if (n == 0) {
        total = 0.0;
        route.clear();
```

```
        return false;
    }

    vector<Edge> sorted = edges;
    sort(sorted.begin(), sorted.end(), [&](const Edge &a, const Edge &b) {
        if (abs(a.w - b.w) > 1e-9) {
            return a.w < b.w;
        }
        if (nodes[a.u] != nodes[b.u]) {
            return nodes[a.u] < nodes[b.u];
        }
        return nodes[a.v] < nodes[b.v];
    });

    DSU dsu;
    dsu.init(n);
    vector<Edge> mst;
    vector<string> steps;
    vector<int> in_tree(n, 0);
    vector<string> current_set{nodes[0]};
    in_tree[0] = 1;
    steps.push_back("起点 " + format_set(current_set));
    total = 0.0;
```

```
for (size_t i = 0; i < sorted.size(); ++i) {
    // 边按权值从小到大尝试加入生成树。
    int u = sorted[i].u;
    int v = sorted[i].v;
    double w = sorted[i].w;
    if (dsu.merge(u, v)) {
        mst.push_back(sorted[i]);
        total += w;
        in_tree[u] = in_tree[v] = 1;
        current_set.clear();
        for (int k = 0; k < n; ++k) {
            if (in_tree[k]) {
                current_set.push_back(nodes[k]);
            }
        }
        sort(current_set.begin(), current_set.end());
        stringstream info;
        info << fixed << setprecision(3)
             << "选择边 (" << nodes[u] << " - " << nodes[v] << ") 权重 "
             << w << " -> 当前节点集合 " << format_set(current_set);
        steps.push_back(info.str());
        if (static_cast<int>(mst.size()) == n - 1) {
            break;
        }
```

```
            }
        }

        vector<vector<int>> adj(n);
        for (size_t i = 0; i < mst.size(); ++i) {
            int u = mst[i].u;
            int v = mst[i].v;
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
        vector<int> vis(n, 0);
        vector<int> order;
        dfs(0, adj, nodes, vis, order);

        route = build_route(nodes, order);
        write_steps(join(out_dir, "steps.txt"), steps, total);
        write_route(join(out_dir, "route.txt"), route, total);
        write_svg(nodes, edges, mst, join(out_dir, "mst.svg"));
        return true;
}
```

edges_readable.csv：

```
source,target,value
里仁,西区 5 组团,0.45
里仁,大食堂,0.48
```

里仁,燕园餐厅,0.85

里仁,西一,0.62

里仁,西二,0.55

里仁,西三,0.47

里仁,西五,1.0

里仁,电气楼,0.53

里仁,理学院,1.25

里仁,艺术与设计学院,1.05

里仁,西里西亚,1.11

里仁,信院,1.3

西区 5 组团,大食堂,0.12

西区 5 组团,燕园餐厅,0.55

西区 5 组团,西二,0.51

西区 5 组团,西三,0.48

西区 5 组团,西五,0.9

西区 5 组团,电气楼,0.49

西区 5 组团,理学院,1.23

西区 5 组团,艺术与设计学院,0.9

西区 5 组团,西里西亚,1.09

西区 5 组团,信院,1.2

大食堂,燕园餐厅,0.45

大食堂,西一,0.45

大食堂,西二,0.4

大食堂,西三,0.38

大食堂,西五,0.8

大食堂,电气楼,0.38

大食堂,理学院,1.1

大食堂,艺术与设计学院,0.79

大食堂,西里西亚,0.98

大食堂,信院,1.1

燕园餐厅,西一,0.33

燕园餐厅,西二,0.36

燕园餐厅,西三,0.42

燕园餐厅,西五,0.48

燕园餐厅,电气楼,0.2

燕园餐厅,理学院,0.78

燕园餐厅,艺术与设计学院,0.38

燕园餐厅,西里西亚,0.65

燕园餐厅,信院,0.74

西一,西二,0.075

西一,西三,0.14

西一,西五,0.36

西一,电气楼,0.16

西一,理学院,0.68

西一,艺术与设计学院,0.42

西一,西里西亚,0.54

西一,信院,0.7

西二,西三,0.067

西二,西五,0.43

西二,电气楼,0.15

西二,理学院,0.74

西二,艺术与设计学院,0.5

西二,西里西亚,0.6

西二,信院,0.77

西三,西五,0.49

西三,电气楼,0.21

西三,理学院,0.8

西三,艺术与设计学院,0.57

西三,西里西亚,0.66

西三,信院,0.84

西五,电气楼,0.41

西五,理学院,0.32

西五,艺术与设计学院,0.19

西五,西里西亚,0.19

西五,信院,0.35

电气楼,理学院,0.74

电气楼,艺术与设计学院,0.42

电气楼,西里西亚,0.61

电气楼,信院,0.15

理学院,艺术与设计学院,0.4

理学院,西里西亚,0.12

理学院,信院,0.15

艺术与设计学院,西里西亚,0.31

艺术与设计学院,信院,0.36

西里西亚,信院,0.2

adjacency_matrix.csv：

,信院,大食堂,燕园餐厅,理学院,电气楼,艺术与设计学院,西一,西三,西二,西五,西区 5组团,西里西亚,里仁

信院 ,0.000000,1.100000,0.740000,0.150000,0.150000,0.360000,0.700000,0.840000,0.770000,0.350000,1.200000,0.200000,1.300000

大食堂 ,1.100000,0.000000,0.450000,1.100000,0.380000,0.790000,0.450000,0.380000,0.400000,0.800000,0.120000,0.980000,0.480000

燕园餐厅 ,0.740000,0.450000,0.000000,0.780000,0.200000,0.380000,0.330000,0.420000,0.360000,0.480000,0.550000,0.650000,0.850000

理学院 ,0.150000,1.100000,0.780000,0.000000,0.740000,0.400000,0.680000,0.800000,0.740000,0.320000,1.230000,0.120000,1.250000

电气楼 ,0.150000,0.380000,0.200000,0.740000,0.000000,0.420000,0.160000,0.210000,0.150000,0.410000,0.490000,0.610000,0.530000

艺术与设计学院 ,0.360000,0.790000,0.380000,0.400000,0.420000,0.000000,0.420000,0.570000,0.500000,0.190000,0.900000,0.310000,1.050000

西一 ,0.700000,0.450000,0.330000,0.680000,0.160000,0.420000,0.000000,0.140000,0.075000,0.360000,INF,0.540000,0.620000

西三 ,0.840000,0.380000,0.420000,0.800000,0.210000,0.570000,0.140000,0.000000,0.067000,0.490000,0.480000,0.660000,0.470000

西二 ,0.770000,0.400000,0.360000,0.740000,0.150000,0.500000,0.075000,0.067000,0.000000,0.430000,0.510000,0.600000,0.550000

西五,0.350000,0.800000,0.480000,0.320000,0.410000,0.190000,0.360000,0.490000,0.430000,0.000000,0.900000,0.190000,1.000000

西区5组团,1.200000,0.120000,0.550000,1.230000,0.490000,0.900000,INF,0.480000,0.510000,0.900000,0.000000,1.090000,0.450000

西里西亚,0.200000,0.980000,0.650000,0.120000,0.610000,0.310000,0.540000,0.660000,0.600000,0.190000,1.090000,0.000000,1.110000

里仁,1.300000,0.480000,0.850000,1.250000,0.530000,1.050000,0.620000,0.470000,0.550000,1.000000,0.450000,1.110000,0.000000